# 🏗 Project Report: Highly Available Web Application on AWS

## Implementation: Manual Setup & Terraform Automation (IaC)

---

## 📌 Project Overview

This project demonstrates a production-ready **3-Tier Architecture** on AWS. The infrastructure was initially built manually to understand resource dependencies and then successfully transitioned into **Terraform** for automation and consistency.

### 🔧 Key Technical Features

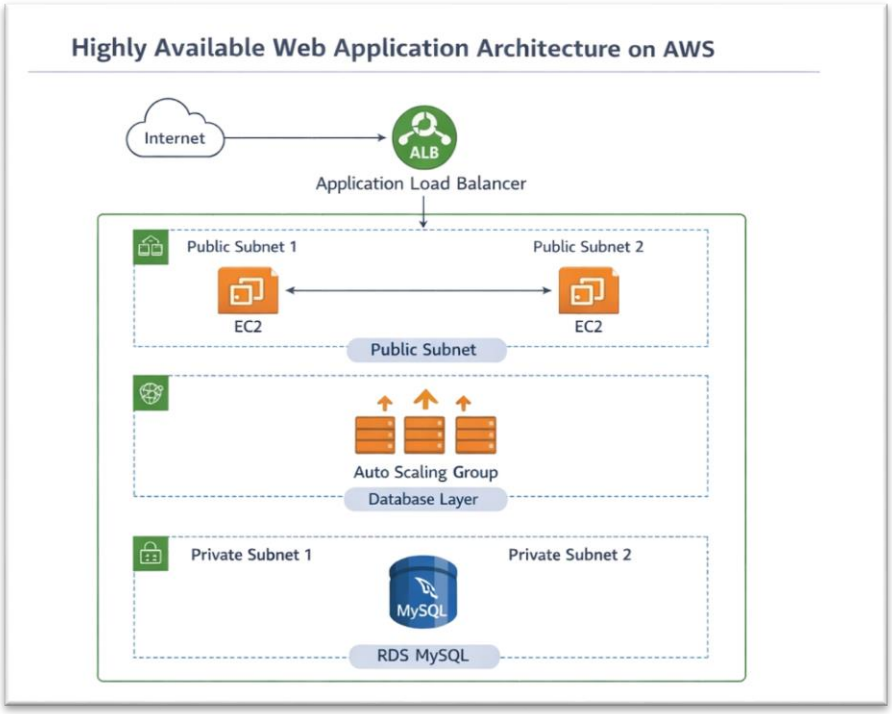
- **High Availability:** Multi-AZ deployment for Compute and Database layers.
- **Scalability:** Auto Scaling Group (ASG) and Application Load Balancer (ALB).
- **Security:** Private subnet isolation for the database and strict Security Group rules.
- **Infrastructure as Code:** Existing manual resources imported and managed via Terraform.

---

## 🏛 Architecture Design

The application is divided into three logical layers:

1. **Web Layer:** Public-facing ALB distributing traffic.
2. **App Layer:** EC2 instances in public subnets managed by an Auto Scaling Group.
3. **Database Layer:** RDS MySQL instance secured in private subnets.




---

# Network & Security Configuration

## VPC & Subnets

- **VPC:** 10.0.0.0/16 (**ANI-HA-VPC**)
- **Public Subnets:** 10.0.1.0/24 (AZ-a), 10.0.2.0/24 (AZ-b) — *For Web/ALB*
- **Private Subnets:** 10.0.101.0/24 (AZ-a), 10.0.102.0/24 (AZ-b) — *For RDS*

## Security Group Rules

| SG Name | Port | Protocol | Source |
|---|---|---|---|
| **ANI-Web-SG** | 80, 22 | TCP | 0.0.0.0/0 |
| **ANI-ALB-SG** | 80 | TCP | 0.0.0.0/0 |
| **ANI-RDS-SG** | 3306 | TCP | ANI-Web-SG |

---

# Compute & Scaling

- **EC2 Instances:** 2x t3.micro running Amazon Linux 2 with Apache Web Server.
- **ALB:** ANI-HA-ALB (Internet-facing) with a listener on Port 80.
- **ASG:** ANI-ASG configured with a Launch Template to maintain a desired capacity of 2 instances.

---

# ⚙ Terraform IaC Implementation

The project successfully moved from manual clicks to automated code using the `terraform import` workflow.

## Resource Import Summary

Bash
```
# Example Import Commands Executed:

terraform import aws_vpc.my_vpc vpc-08f1d66138f91f57d
terraform import aws_subnet.public_1 subnet-00a49659cbc865308
terraform import aws_security_group.web_sg sg-09323f0dda300dc88
terraform import aws_instance.web1 i-048aba6e45a49a4e4
terraform import aws_db_instance.my_rds ani-db
```

## Infrastructure Summary

| Resource | Count |
|---|---|
| EC2 Instances | 2 |
| RDS Instances | 1 |
| Load Balancers | 1 |
| Subnets | 4 |

---

# 🚀 Project Outcome

- **Fault Tolerance:** Architecture remains operational even if one Availability Zone fails.
- **IaC Maturity:** Successfully eliminated "Manual Drift" by syncing the AWS console with Terraform state.
- **Security:** Backend database is completely unreachable from the public internet.

---

# 🌟 Future Improvements

- Implement HTTPS using **AWS Certificate Manager (ACM)**.
- Add **CloudWatch Alarms** for automated scaling based on CPU metrics.
- Integrate a **CI/CD Pipeline** using GitHub Actions for automated Terraform applies.

---

**Prepared by:** Anitha RS
**Date:** 05-04-2026